

Serialización

Programación III - UNGS

Arquitectura en tres capas

- Hasta ahora, tenemos herramientas para construir:

Arquitectura en tres capas

- Hasta ahora, tenemos herramientas para construir:
 1. Una **interfaz de usuario** por consola (típicamente desde la función **main**) o bien con un form desde WindowBuilder.

Arquitectura en tres capas

- Hasta ahora, tenemos herramientas para construir:
 1. Una **interfaz de usuario** por consola (típicamente desde la función **main**) o bien con un form desde WindowBuilder.
 2. Un **modelo de objetos** (dado por un diagrama de clases) que representa la lógica del negocio y cuyas funciones son ejecutadas desde la función que implementa la interfaz.

Arquitectura en tres capas

- Hasta ahora, tenemos herramientas para construir:
 1. Una **interfaz de usuario** por consola (típicamente desde la función **main**) o bien con un form desde WindowBuilder.
 2. Un **modelo de objetos** (dado por un diagrama de clases) que representa la lógica del negocio y cuyas funciones son ejecutadas desde la función que implementa la interfaz.
 3. Ninguna capacidad de **almacenamiento permanente**.

Arquitectura en tres capas

- Hasta ahora, tenemos herramientas para construir:
 1. Una **interfaz de usuario** por consola (típicamente desde la función **main**) o bien con un form desde WindowBuilder.
 2. Un **modelo de objetos** (dado por un diagrama de clases) que representa la lógica del negocio y cuyas funciones son ejecutadas desde la función que implementa la interfaz.
 3. Ninguna capacidad de **almacenamiento permanente**.
- La forma más sencilla de implementar almacenamiento permanente en un sistema orientado a objetos es a través de la **persistencia** de los objetos, habitualmente por medio de un mecanismo de **serialización**.

Serialización

- Llamamos **serialización** al proceso por el cual se convierte un objeto o una estructura de datos a un formato que puede ser almacenado (en un archivo o transmitido a través de una red).

Serialización

- Llamamos **serialización** al proceso por el cual se convierte un objeto o una estructura de datos a un formato que puede ser almacenado (en un archivo o transmitido a través de una red).
- La intención es que el mismo objeto pueda ser “recreado” nuevamente a partir de la información almacenada, en la misma o en otra computadora.

Serialización

- Llamamos **serialización** al proceso por el cual se convierte un objeto o una estructura de datos a un formato que puede ser almacenado (en un archivo o transmitido a través de una red).
- La intención es que el mismo objeto pueda ser “recreado” nuevamente a partir de la información almacenada, en la misma o en otra computadora.
- En Java la serialización es automática, siempre que se identifique la clase como serializable implementando la interface **Serializable**.

La interfaz Serializable

```

1  import java.io.Serializable;
2
3  public class Persona implements Serializable
4  {
5      private String nombre;
6      private String DNI;
7      private int edad;
8      private static final long serialVersionUID = 1L;
9
10     public Persona(String n, String d, int e)
11     {
12         nombre = n;
13         DNI = d;
14         edad = e;
15     }
16     ...
17 }
```

La interfaz Serializable

- Serializamos a un archivo tres instancias de la clase. Para esto, tenemos que incluir las siguientes bibliotecas:

La interfaz Serializable

- Serializamos a un archivo tres instancias de la clase. Para esto, tenemos que incluir las siguientes bibliotecas:

```
1 import java.io.FileOutputStream;
2 import java.io.ObjectOutputStream;
```

La interfaz Serializable

- Serializamos a un archivo tres instancias de la clase. Para esto, tenemos que incluir las siguientes bibliotecas:

```

1 import java.io.FileOutputStream;
2 import java.io.ObjectOutputStream;
```

- Creamos las tres personas ...

La interfaz Serializable

- Serializamos a un archivo tres instancias de la clase. Para esto, tenemos que incluir las siguientes bibliotecas:

```
1 import java.io.FileOutputStream;
2 import java.io.ObjectOutputStream;
```

- Creamos las tres personas ...

```
1 public static void main2(String[] args)
2 {
3     Persona p1 = new Persona("Pepe", "28655654", 32);
4     Persona p2 = new Persona("Alicia", "32251251", 28);
5     Persona p3 = new Persona("Alberto", "17852584", 51);
6     ...
```

La interfaz Serializable

- ... y serializamos al archivo:

```

1  try // Debe estar en un try/catch
2  {
3      FileOutputStream fos = new FileOutputStream(" personas.txt" );
4      ObjectOutputStream out = new ObjectOutputStream(fos);
5      out.writeObject(p1);
6      out.writeObject(p2);
7      out.writeObject(p3);
8      out.close();
9  }
10 catch (Exception ex)
11 {
12     ex.printStackTrace();
13 }
```

La interfaz Serializable

- Realizamos un procedimiento similar para leer las tres personas desde el archivo:

```

1  try // Debe estar en un try/catch
2  {
3      FileInputStream fis = new FileInputStream("personas.txt");
4      ObjectInputStream in = new ObjectInputStream(fis);
5
6      p1 = (Persona) in.readObject();
7      p2 = (Persona) in.readObject();
8      p3 = (Persona) in.readObject();
9
10     in.close();
11 }
12 catch (Exception ex) { ... }
```

La interfaz Serializable

- Podemos serializar estructuras de datos, haciendo que todas las clases de la estructura sean serializables.

La interfaz Serializable

- Podemos serializar estructuras de datos, haciendo que todas las clases de la estructura sean serializables.
- Por ejemplo, tenemos una clase que implementa un conjunto de personas.

La interfaz Serializable

- Podemos serializar estructuras de datos, haciendo que todas las clases de la estructura sean serializables.
- Por ejemplo, tenemos una clase que implementa un conjunto de personas.

```

1  public class Padron implements Serializable
2  {
3      private static final long serialVersionUID = 1L;
4      private LinkedList<Persona> personas;
5
6      public Padron()
7      {
8          personas = new LinkedList<Persona>();
9      }
10     ...
    
```

La interfaz Serializable

- Serializamos el padrón con el siguiente código:

```

1  Padron p = new Padron();
2  ...
3
4  try // Debe estar en un try/catch
5  {
6      FileOutputStream fos = new FileOutputStream("padron.txt");
7      ObjectOutputStream out = new ObjectOutputStream(fos);
8      out.writeObject(p);
9      out.close();
10 }
11 catch (Exception ex)
12 {
13     ex.printStackTrace();
14 }
```

La interfaz Serializable

- ... y lo leemos desde el archivo:

```

1  Padron p = null;
2  try
3  {
4      FileInputStream fis = new FileInputStream("padron.txt");
5      ObjectInputStream in = new ObjectInputStream(fis);
6      p = (Padron) in.readObject();
7      in.close();
8  }
9  catch (Exception ex)
10 {
11     ex.printStackTrace();
12 }
```

La interfaz Serializable

- Para que **Padron** sea serializada correctamente, la clase **Persona** (y todas las clases auxiliares involucradas) deben ser serializables.

La interfaz Serializable

- Para que **Padron** sea serializada correctamente, la clase **Persona** (y todas las clases auxiliares involucradas) deben ser serializables.
- Si no queremos que algún miembro sea serializado, se debe agregar la palabra reservada **transient** a su definición:

La interfaz Serializable

- Para que **Padron** sea serializada correctamente, la clase **Persona** (y todas las clases auxiliares involucradas) deben ser serializables.
- Si no queremos que algún miembro sea serializado, se debe agregar la palabra reservada **transient** a su definición:

```

1  public class Persona implements Serializable
2  {
3      private String nombre;
4      private String DNI;
5      transient private int edad;
6      private static final long serialVersionUID = 1L;
    
```

La interfaz Serializable

- Para que **Padron** sea serializada correctamente, la clase **Persona** (y todas las clases auxiliares involucradas) deben ser serializables.
- Si no queremos que algún miembro sea serializado, se debe agregar la palabra reservada **transient** a su definición:

```

1  public class Persona implements Serializable
2  {
3      private String nombre;
4      private String DNI;
5      transient private int edad;
6      private static final long serialVersionUID = 1L;
    
```

- Al momento de leer los objetos, las edades serán cero.

JSON

- JSON (*JavaScript Object Notation*) es un formato liviano para intercambio de datos entre aplicaciones.

JSON

- JSON (*JavaScript Object Notation*) es un formato liviano para intercambio de datos entre aplicaciones.
 1. Formato de texto, similar a XML.

JSON

- JSON (*JavaScript Object Notation*) es un formato liviano para intercambio de datos entre aplicaciones.
 1. Formato de texto, similar a XML.
 2. Fácil de generar e interpretar, y fácil de parsear.

JSON

- JSON (*JavaScript Object Notation*) es un formato liviano para intercambio de datos entre aplicaciones.
 1. Formato de texto, similar a XML.
 2. Fácil de generar e interpretar, y fácil de parsear.
 3. www.json.org

JSON

- JSON (*JavaScript Object Notation*) es un formato liviano para intercambio de datos entre aplicaciones.
 1. Formato de texto, similar a XML.
 2. Fácil de generar e interpretar, y fácil de parsear.
 3. www.json.org
-

```

1  {
2    "nombre": "Juan Perez",
3    "nacionalizado": true,
4    "edad": 25,
5    "direccion": {
6      "calle": "J. M. Gutierrez",
7      "numero": "1150"
8    }
9  }
```

JSON

- Permite representar arreglos y listas:

```

1  {
2    "nombre": "Juan Perez",
3    "telefonos": [
4      {
5        "tipo": "fijo",
6        "numero": "4675-4781"
7      },
8      {
9        "tipo": "celular",
10       "numero": "15-4535-7651"
11     }
12   ],
13   "conyuge": null,
14 }
```

JSON

- Podemos generar un archivo JSON asociado con una clase, representando la estructura de la clase.

JSON

- Podemos generar un archivo JSON asociado con una clase, representando la estructura de la clase.

```

1  import com.google.gson.Gson;
2  import com.google.gson.GsonBuilder;
3
4  public class ArchivoJSON
5  {
6      private String nombre;
7      private int edad;
8      private ArrayList<String> telefonos;
9
10     ...
11 }
```

JSON

- Para generar un archivo JSON ...

JSON

- Para generar un archivo JSON ...

```

1  public void generarJSON(String archivo)
2  {
3      Gson gson = new GsonBuilder().setPrettyPrinting().create();
4      String json = gson.toJson(this);
5
6      try
7      {
8          FileWriter writer = new FileWriter(archivo);
9          writer.write(json);
10         writer.close();
11     }
12     catch(Exception e) { ... }
13 }
```

JSON

- Para leer un archivo JSON ...

JSON

- Para leer un archivo JSON ...

```

1  public static ArchivoJSON leerJSON(String archivo)
2  {
3      Gson gson = new Gson();
4      ArchivoJSON ret = null;
5
6      try
7      {
8          BufferedReader br = new BufferedReader(new FileReader(archivo));
9          ret = gson.fromJson(br, ArchivoJSON.class);
10     }
11     catch (Exception e) { ... }
12     return ret;
13 }
```

Usos de la serialización

- Lograr **persistencia** de los objetos de la aplicación. De esta forma se pueden tener aplicaciones con datos permanentes entre ejecuciones.

Usos de la serialización

- Lograr **persistencia** de los objetos de la aplicación. De esta forma se pueden tener aplicaciones con datos permanentes entre ejecuciones.
 - Es importante observar que un mecanismo más adecuado de persistencia se obtiene almacenando los objetos en una base de datos, sobre todo para el manejo de **conurrencia**.

Usos de la serialización

- Lograr **persistencia** de los objetos de la aplicación. De esta forma se pueden tener aplicaciones con datos permanentes entre ejecuciones.
 - Es importante observar que un mecanismo más adecuado de persistencia se obtiene almacenando los objetos en una base de datos, sobre todo para el manejo de **conurrencia**.
- **Enviar un conjunto de objetos** de un proceso a otro. El proceso emisor guarda los objetos en un archivo y el proceso receptor, sabiendo de qué tipo son los objetos, los lee desde el archivo.

Usos de la serialización

- Lograr **persistencia** de los objetos de la aplicación. De esta forma se pueden tener aplicaciones con datos permanentes entre ejecuciones.
 - Es importante observar que un mecanismo más adecuado de persistencia se obtiene almacenando los objetos en una base de datos, sobre todo para el manejo de **conurrencia**.
- **Enviar un conjunto de objetos** de un proceso a otro. El proceso emisor guarda los objetos en un archivo y el proceso receptor, sabiendo de qué tipo son los objetos, los lee desde el archivo.
- Hacer una **copia de seguridad** de los objetos, especialmente para detectar cambios en los datos.